D&D Game Master Assistant — Vibe-Coding Capstone Plan

Build & Deploy Roadmap · Google × Kaggle 5-Day AI Agents · Prepared 2026-06-23

Contents

Context

This plan is the **prompt-by-prompt roadmap** for building, evaluating, and (finally) deploying a **Dungeons & Dragons Game Master Assistant** as the submission for the *Vibe Coding Agents Capstone Project* (Google × Kaggle 5-Day AI Agents course). The whole app is built **purely via vibe prompts** in Antigravity / Claude Code.

Problem it solves: During live D&D play, a Game Master juggles rules arbitration, on-the-fly NPC voices, and session bookkeeping simultaneously. This app is a multi-agent co-GM that handles three concrete pain points — **rules/combat math, NPC dialogue grounded in campaign lore, and session recapping** — while staying safe and remembering campaign state across sessions.

Why agents (central to scoring): the task is genuinely multi-skill and stateful — a single prompt can't safely route a rules question vs. an NPC line vs. a recap, ground each in the right lore file, mutate party HP, and persist the campaign. A guardrail → supervisor → specialist triad with shared tools/state is the natural fit.

Current repo state (greenfield app, rich data): no agent/frontend/API code exists yet. We DO have: - `docs/KNOWLEDGE.md` — 221-line index of 200+ Tomb of Annihilation (ToA) markdown lore files. - `docs/Tomb-of-Annihilation/**` — the lore files themselves + `Appendix D.csv` (828 creature stat blocks). - `assets/Tomb-of-Annihilation/ASSETS.md` — 117 image URLs (maps, NPC portraits, handouts). - `conversation-egress/` — working MongoDB plumbing (pymongo, `vibe_coding` DB) + Stop-hook conversation logging. - Google ADK + Agents-CLI skills already enabled in `.claude/settings.json`.

Decisions locked (from user)

1. **Knowledge retrieval = manifest-router pattern** (NOT embeddings/vector DB): feed `KNOWLEDGE.md` + `ASSETS.md` to a router agent → it picks relevant file paths from the user query → a second agent calls an

MCP tool to read those files → answers the GM. Selected paths double as `lore_provenance`.

2. **Two Cloud Run services** (ADK backend + Next.js frontend) — but **deployment artifacts only in the final phase**. Everything is built and tested **locally** until then. (Rubric confirms live deploy is NOT required for judging.)
3. **Full build, sequential** (no early throwaway MVP): complete backend → quality → frontend → deploy.
4. **Stack:** Python + Google ADK (Gemini models) for agents; tools are **plain ADK FunctionTools (no MCP/FastMCP)** served via ADK 2.0's `get_fast_api_app` (FastAPI) — agents call them as tools, and the UI calls the same functions through custom FastAPI routes on that app; MongoDB (`vibe_coding` DB) for state/memory; Next.js + React (TS) + Tailwind for UI.
5. **No MCP server:** per the stack decision we do NOT build a FastMCP/MCP server. We still cover 5/6 required concepts (≥3 needed), so this is safe for scoring.

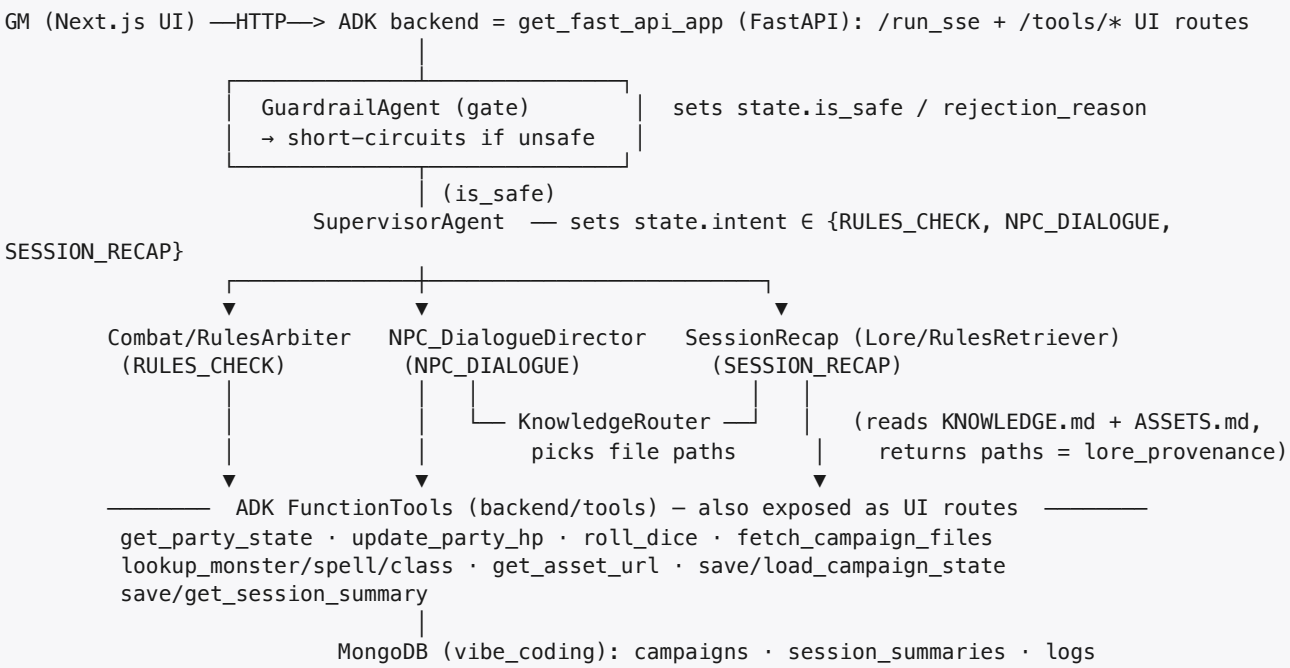
How this maps to the rubric (target the points)

Rubric item	Pts	Where in this plan
Technical Implementation (architecture, code quality, meaningful agents, clever tool use, **code comments**)	50	Phases 2–9
Documentation (README: problem/solution/architecture/setup + diagrams)	20	Phase 10
Core Concept & Value (agents clear/meaningful/central)	10	Whole design; articulated Phase 10
YouTube video ≤5 min (problem, why-agents, architecture, demo, the build)	10	Phase 10 assets
Writeup	10	Phase 10

Required concepts — we hit 5/6 (need ≥3; MCP intentionally dropped per stack decision): - Multi-agent ADK → *Code* (Phase 3) - Security features → *Code/Video* (GuardrailAgent, Phase 3/7) - Agent skills / Agents-CLI → *Code/Video* (used throughout: scaffold, eval) - Antigravity → *Video* (we vibe-code in Antigravity; show it in the build segment) - Deployability → *Video* (Phase 11 + demo)

Non-negotiables baked in: no API keys/passwords in code (env + Secret Manager); meaningful code comments on agents/tools/callbacks; architecture diagram(s) in README.

Target architecture



State keys written for the UI trace panel: `is_safe`, `rejection_reason`, `intent`, `last_agent`, `tools_fired[]`, `lore_provenance[]`, `dice[]`, `party`, `scene`, `initiative[]`.

Target repo layout (deployment files appear ONLY in Phase 11)

```
backend/
  agent/            # ADK agents: guardrail, supervisor, arbiter, npc, recap, knowledge_router
  tools/            # plain ADK FunctionTools (party, dice, fetch_files, lookups, asset, persistence)
  app.py            # get_fast_api_app(...) -> FastAPI; + custom /tools/* & /party/* UI routes
  data/open5e/      # JSON dumps (spells, monsters, classes)
  persistence/      # MongoDB session/campaign repo
  eval/             # *.evalset.json + eval configs
  .env.example      # NO secrets
frontend/           # Next.js + React (TS) + Tailwind (3 panels)
docs/ , assets/     # existing knowledge base (unchanged)
README.md           # Phase 10
# Dockerfiles / cloudbuild / Cloud Run config -> added in Phase 11 ONLY
```

Phased vibe-prompt roadmap

> Each phase = a milestone with copy-pasteable prompts and a **local** verification gate. Do not advance until the gate passes. Prompts assume Antigravity/Claude Code with the project's Google-ADK skills enabled.

Part A — Backend (local)

Phase 0 — Scaffold & repo layout

Goal: ADK project skeleton + repo structure, no business logic yet.

```
@~/.claude/skills/google-agents-cli-scaffold/SKILL.md
Scaffold a new Google ADK Python project under backend/ named "gm_assistant" using Gemini.
Create the repo layout: backend/agent, backend/mcp_server, backend/data/open5e, backend/persistence,
backend/eval, and an empty frontend/ placeholder. Add backend/.env.example with MONGODB_URI,
DB_NAME=vibe_coding,
and GOOGLE_API_KEY placeholders (NO real secrets). Do NOT add any Docker/Cloud Run files yet — local
dev only.
Wire a Makefile/justfile with local run targets. Add concise module docstrings.
```

Gate: `adk run` (or `adk web`) launches the empty agent locally; `.env.example` has no real secrets.

Phase 1 — Data ingestion (Open5e + verify manifests)

Goal: local JSON knowledge for rules; confirm lore manifests load.

```
Write backend/data/fetch_open5e.py that pulls spells, monsters, and classes from the Open5e API
(https://api.open5e.com) and writes paginated results to
backend/data/open5e/{spells,monsters,classes}.json.
Make it idempotent and offline-friendly (skip if files exist, --refresh to force). Add a tiny loader
module
with lookup-by-name helpers. Also add a loader that reads docs/KNOWLEDGE.md and assets/Tomb-of-
Annihilation/ASSETS.md
as plain text (these are the retrieval manifests). Comment the rate-limiting and pagination logic.
```

Gate: JSON files exist and `lookup_monster("froghemoth")`, `lookup_spell("fireball")` return data locally.

Phase 2 — Tools + ADK FastAPI app (`get_fast_api_app`, MongoDB-backed)

Goal: all GM tools as plain ADK FunctionTools, served by ADK 2.0's FastAPI app and callable by BOTH agents and the UI.

```
@~/.claude/skills/google-agents-cli-adk-code/SKILL.md
Implement backend/tools/ as plain Python functions wrapped as ADK FunctionTools (NO MCP / FastMCP),
each with typed
args, docstrings, and inline comments:
- get_party_state(campaign_id) -> party HP/conditions, scene, initiative (from MongoDB).
- update_party_hp(campaign_id, character, delta|new_hp) -> persists and returns updated party.
- roll_dice(notation) -> structured result for "1d20+5","2d6","1d100" (value, rolls, crit flags) for UI
animation.
- fetch_campaign_files(paths[]) -> reads docs/Tomb-of-Annihilation markdown files, returns text +
provenance.
- lookup_monster/lookup_spell/lookup_class(name) -> from backend/data/open5e.
- get_asset_url(description) -> resolve image URL via ASSETS.md base + file.
- save_campaign_state / load_campaign_state(campaign_id); save_session_summary /
get_session_history(campaign_id).
Create backend/app.py that builds the server with ADK's get_fast_api_app(...) (FastAPI) to serve the
agent graph
(/run, /run_sse, session endpoints). Then ADD custom FastAPI routes on that SAME app so the UI can call
tools
DIRECTLY without invoking an agent — e.g. GET /party/{campaign_id}, POST /tools/roll_dice, POST
/tools/update_hp —
each delegating to the same tool functions (single source of truth). Back state/memory with MongoDB
(vibe_coding DB):
collections campaigns, session_summaries. Reuse the pymongo pattern from conversation-egress/. Read
MONGODB_URI from
env only. Add a smoke-test script that calls every tool function and every UI route.
```

Gate: the FastAPI app (get_fast_api_app) starts locally; smoke test exercises every tool; UI routes (/party, /tools/roll_dice, /tools/update_hp) return JSON; HP update + session summary round-trip to MongoDB.

Phase 3 — Agent triad (Guardrail → Supervisor → specialists) *(Multi-agent ADK + Security → Code)*

Goal: the full ADK graph wired to MCP tools and session state.

```
@~/.claude/skills/google-agents-cli-adk-code/SKILL.md
Implement the ADK multi-agent graph in backend/agent, binding the backend/tools FunctionTools directly
to the agents:
1. GuardrailAgent as a guardrail (before_agent/before_model callback on the root): detect prompt
injection /
   out-of-scope, set state.is_safe and state.rejection_reason, and SHORT-CIRCUIT with a safe refusal if
unsafe.
2. SupervisorAgent: classify the GM input into intent ∈ {RULES_CHECK, NPC_DIALOGUE, SESSION_RECAP},
write
   state.intent, and delegate to the matching specialist (ADK sub_agents / transfer).
3. Combat/RulesArbiter (RULES_CHECK): bind get_party_state, update_party_hp, roll_dice,
lookup_monster/spell/class;
   compute outcomes and persist HP. Show its math.
4. NPC_DialogueDirector (NPC_DIALOGUE): ground NPC reactions via the knowledge subsystem (Phase 4).
5. SessionRecap / Lore-RulesRetriever (SESSION_RECAP): summarize the scene, format GM notes, suggest
the next scene.
Every agent appends its name to state.last_agent and tools to state.tools_fired for observability.
Heavy code
comments on routing, callbacks, and state contracts. Pick current Gemini models per the google-agents-
cli-workflow skill.
```

Gate: `adk web` locally — a rules question updates HP in MongoDB; routing lands on the correct specialist; an injection attempt is blocked with a visible `rejection_reason`.

Phase 4 — Knowledge routing subsystem (manifest-router pattern)

Goal: the user's exact retrieval design, shared by NPC + Recap agents.

```
Add KnowledgeRouterAgent: its context is the full text of docs/KNOWLEDGE.md and assets/.../ASSETS.md.
Given the
GM query, it outputs a STRUCTURED list of the most relevant lore file paths (and optionally an asset
description).
Then the calling specialist (NPC_DialogueDirector or SessionRecap) calls the fetch_campaign_files MCP
tool with
those paths, grounds its answer in the returned text, and writes the chosen paths to
state.lore_provenance.
Compose this as: KnowledgeRouter (file picker) -> fetch_campaign_files -> grounded answer. Keep the
router cheap/fast.
Comment why we use a manifest router instead of vector RAG (small, well-indexed corpus; exact
provenance).
```

Gate: "What faction controls the docks in Port Nyanzaru?" returns a grounded answer whose `lore_provenance` points at the correct `Ch-1-Port Nyanzaru/*.md` file(s).

Phase 5 — Session state + MongoDB persistence (campaign resume) *(Context engineering)*

Goal: durable, resumable campaigns + evolving session history.

```
Add a MongoDB-backed persistence layer so a campaign can be resumed: load_campaign_state hydrates ADK
session state
(party, scene, initiative, history) at session start; save_campaign_state snapshots it. After each
SESSION_RECAP,
append an evolving summary (decisions, party actions, next-scene suggestions) to session_summaries so
the assistant
remembers prior GM decisions. Expose start/resume by campaign_id. Comment the state lifecycle and the
snapshot points.
```

Gate: Kill and restart the backend, resume the same `campaign_id`, and prior HP + last recap are intact.

Part B — Quality (local)

Phase 6 — Evaluation *(Agent quality / Agents-CLI eval)*

Goal: measurable quality with ADK eval; iterate the flywheel.

```
@~/.claude/skills/google-agents-cli-eval/SKILL.md
Create backend/eval/*.evalset.json covering: RULES_CHECK (HP math + correct tool calls), NPC_DIALOGUE
(answer is
grounded — lore_provenance non-empty and on-topic), SESSION_RECAP (recap mentions key party actions),
and ROUTING
(intent classification accuracy). Add adversarial GUARDRAIL cases (prompt injection, jailbreaks) that
MUST be
blocked. Run the eval, summarize pass/fail, and fix the top failures. Document the eval methodology and
results.
```

Gate: `adk eval` runs locally; routing + guardrail suites pass; failures triaged and addressed.

Phase 7 — Security hardening *(Security features → Code/Video)*

Goal: strengthen the guardrail story + secrets hygiene + threat model.

```
@~/.claude/skills/stride-threat-model/SKILL.md
Harden the GuardrailAgent (input + output filtering, refusal copy) and add more injection/abuse eval
cases.
Confirm NO secrets in code (MONGODB_URI/GOOGLE_API_KEY from env only) and add a secrets check. Produce
a short
STRIDE threat-model section for the README (spoofing/injection, tampering with HP/state, info
disclosure of lore,
etc.) with mitigations. Comment the security-relevant code paths.
```

Gate: All adversarial eval cases blocked; repo grep for keys is clean; STRIDE notes drafted.

Part C — Frontend (local)

Phase 8 — Next.js 3-panel UI shell

Goal: the GM cockpit (TypeScript + Tailwind), static/mocked first.

```
Create frontend/ as a Next.js (App Router) + React + TypeScript + Tailwind app with a 3-panel layout:
- Panel 1 "Party & Character State": party HP, current scene, initiative order; quick-action buttons
  "Roll Initiative", "Update HP", "Summarize Scene".
- Panel 2 "Session Transcript": a clean chat console (GM input + session_output), a "Play Audio" button
per
  response, and a dice-roll animation component for d20/d100.
- Panel 3 "Agent Trace & Lore Provenance": shows routed agent (e.g. "Routed to: Combat/Rules Arbiter"),
tools fired,
  lore_provenance source files, and guardrail rejection reason when blocked.
Build with mocked data first; clean, table-friendly, dark GM-screen aesthetic. Comment the component
contracts.
```

Gate: `next dev` renders all three panels with mock data and a working dice animation.

Phase 9 — Wire UI ↔ backend (local), dice, TTS, live trace

Goal: end-to-end local loop.

```
Wire the frontend to the local ADK backend (get_fast_api_app / FastAPI). Chat uses /run_sse for the
agent graph; the
quick-action buttons (Roll Initiative, Update HP) and Panel 1 call the direct /tools/* and /party
routes. Send GM
input, stream/receive session_output, and populate: Panel 1 from get_party_state, Panel 2 with the
response +
roll_dice results driving the d20/d100 animation,
Panel 3 from state (last_agent, tools_fired, lore_provenance, is_safe/rejection_reason). Implement
"Play Audio" with
the browser Web Speech API (note Google Cloud TTS as an optional upgrade). Wire quick-action buttons to
the matching
backend flows. Keep the API base URL in an env var (no hardcoding).
```

Gate: Locally, typing a rules query updates Panel 1 HP, animates the die in Panel 2, and Panel 3 shows the arbiter + tools + provenance. An injection attempt shows the rejection in Panel 3.

Part D — Ship

Phase 10 — Docs, diagrams, writeup & video assets *(Documentation 20 + Pitch 30)*

Goal: the submission narrative — heaviest non-code scoring.

```
Write README.md: problem, solution, architecture (with an architecture diagram image + the agent-graph
diagram),
the manifest-router retrieval explanation, local setup instructions, eval results, and the STRIDE
notes. Generate
a clean architecture diagram. Draft a ≤5-minute YouTube video script hitting: Problem → Why agents →
Architecture
(show diagram) → Demo (the 3-panel cockpit in action) → The Build (vibe-coded in Antigravity, ADK, MCP,
Agents-CLI).
Draft the Kaggle writeup (problem/solution/architecture/journey). Double-check no secrets anywhere.
```

Gate: README renders with diagrams; video script + writeup drafted; secrets scan clean.

Phase 11 — Deployment to Cloud Run (LAST — first time we add deploy files) *(Deployability → Video)*

Goal: two Cloud Run services + reproducible deploy docs.

```
@~/.claude/skills/google-agents-cli-deploy/SKILL.md
NOW add deployment artifacts (not before): a Dockerfile for the ADK backend (uvicorn serving
backend/app.py's get_fast_api_app FastAPI) and one for the Next.js
frontend. Deploy both as two separate Cloud Run services. Store MONGODB_URI and GOOGLE_API_KEY in
Secret Manager
(never in code/images). Configure the frontend's API base URL to the backend service URL. Add a
"Deployment"
section to README with exact, reproducible steps. Enable Cloud Trace / basic observability for the
backend.
```

Gate: Both services deploy; the live UI drives the live backend; README deploy steps reproduce it; record the deploy + demo for the video.

End-to-end verification strategy

- **Per-phase local gates** above are the primary checks — do not skip them.
- **Tools:** FunctionTool smoke test + UI route checks (Phase 2) before any agent wiring.
- **Agents:** `adk web` / `adk run` for interactive local testing of routing, HP mutation, guardrail, provenance.

- **Quality:** `adk eval` suites (routing, grounding, guardrail) as a regression gate before frontend work.
- **Full loop:** local Next.js ↔ local FastAPI `get_fast_api_app` (Phase 9) — the real demo, recorded for the video.
- **Deploy repro:** only Phase 11; verify the README steps from a clean shell.

Open items / notes

- **Models:** use current Gemini IDs via the `google-agents-cli-workflow` skill rather than hardcoding (cutoff drift).
- **MongoDB:** local Mongo or Atlas for dev; the same `vibe_coding` DB as `conversation-egress/`. Reuse its pymongo pattern.
- **conversation-egress Stop-hook** already logs sessions to MongoDB — mention it as a bonus observability artifact.
- **Antigravity:** since judging credits Antigravity in the *video*, capture build footage in Antigravity for Phase 10.
- **Scope guard:** `Appendix D.csv` (828 stat blocks) can back a `lookup_creature` tool if Open5e lacks a ToA-specific monster — optional enhancement for the arbiter.